

- 学号: 221240093
- 姓名: 陈力峥
- 邮箱: 221240093@smail.nju.edu.cn

编译

编译采用 `Makefile`，在 `Code` 目录下执行 `make` 即可，随后生成 `parser` 可执行文件。 `Code` 目录下运行 `parser: ./parser [input_IR_file] [output_IR_file]`

实验内容

在实验框架的基础上，完成实验五所有部分:

- 对于数据流分析模式的优化：
 - 公共子表达式消除
 - 无用代码消除
 - 常量传播
 - 循环不变代码外提
 - 归纳变量强度削减

实验细节

前四个主要是补全框架，第五个是实现循环优化。

1. 实验框架中的可用表达式分析：

- 主要补全了 `AvailableExpressionsAnalysis_newBoundaryFact`, `AvailableExpressionsAnalysis_newInitialFact` 以及 `AvailableExpressionsAnalysis_meetInto` 由于可用表达式分析属于前向分析，边界块 (也就是entry块)的初始边界事实是空集(无入边)，非边界块的初始边界事实是全集(后续通过 `meet` 操作不断选出真正的可用表达式)

2. 常量传播优化 关键函数如下，此后 `transferStmt` 中需要注意定义相关的语句进行 `meet` 操作

```
// 常量值表示：UNDEF (未定义)、CONST (常量)、NAC (非常量)
static CPValue meetValue(CPValue v1, CPValue v2) {
    if (v1.kind == UNDEF) return v2;
    if (v2.kind == UNDEF) return v1;
    if (v1.kind == NAC || v2.kind == NAC) return get_NAC();
    if (v1.kind == CONST && v2.kind == CONST) {
        return v1.const_val == v2.const_val ? get_CONST(v1.const_val)
        : get_NAC();
    }
}
```

- 复制传播优化 主要针对赋值语句，当变量被重新定义的时候，其所有之前的定义和使用都被 kill, 随后按照下面的代码记录新的 `def use`

```
// 维护变量间的复制关系
void CopyPropagation_transferStmt(CopyPropagation *t, IR_stmt *stmt,
Fact_def_use *fact) {
    // 处理复制语句: x = y
    if(stmt->stmt_type == IR_ASSIGN_STMT) {
        IR_assign_stmt *assign_stmt = (IR_assign_stmt*)stmt;
        if(!assign_stmt->rs.is_const) {
            IR_var def = assign_stmt->rd, use = assign_stmt->rs.var;
            VCALL(fact->def_to_use, set, def, use);
            VCALL(fact->use_to_def, set, use, def);
        }
    }
}
```

4. 活跃变量分析

- 主要补全了 `LiveVariableAnalysis_isForward`, `LiveVariableAnalysis_meetInto` 以及 `LiveVariableAnalysis_transferStmt` 和 `block_remove_dead_def` 活跃变量分析属于后向分析，初始所有变量均设为不活跃，在 `transferStmt` 中，状态转移方程为 $f_s = use_s \cup (OUT[s] - def_s)$

```
// 后向数据流分析，状态转移方程: f_s = use_s U (OUT[s] - def_s)
void LiveVariableAnalysis_transferStmt(LiveVariableAnalysis *t,
IR_stmt *stmt, Set_IR_var *fact) {
    IR_var def = VCALL(*stmt, get_def);
    if (def != IR_VAR_NONE) {
        VCALL(*fact, delete, def); // kill
    }
    IR_use use = VCALL(*stmt, get_use_vec);
    for (unsigned i = 0; i < use.use_cnt; i++) {
        if (!use.use_vec[i].is_const) {
            VCALL(*fact, insert, use.use_vec[i].var); // gen
        }
    }
}
```

5. 循环优化

- 实现伪代码如下，大致先检测每一个函数中存在的循环，在循环内部的每一块查找循环不变代码和归纳变量，创建循环头上头 block，插入新的语句后更新控制流图。但尚未没有经过严格的测试检查。

```
// 循环不变代码外提
void LoopAnalysis_motion_invariants(LoopAnalysis *t, Loop *loop,
```

```
IR_function *func) {
    // 1. 创建pre-header块
    // 2. 收集循环不变代码
    // 3. 移动代码到pre-header
    // 4. 更新控制流图
}

// 归纳变量强度削减
void LoopAnalysis_reduce_strength(LoopAnalysis *t, Loop *loop,
IR_function *func) {
    // 1. 识别基础归纳变量
    // 2. 识别派生归纳变量
    // 3. 将乘法转换为加法
    // 4. 更新循环控制流
}
```